This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

A Robust and Fast Data Management System for Machine-Learning Research of Tokamaks

Chenguang Wan, Zhi Yu, Xiaojuan Liu, Xinghao Wen, Xi Deng, and Jiangang Li

***Abstract*— In recent years, machine-learning (ML) research methods have received increasing attention in the tokamak community. The conventional database (i.e., MDSplus for tokamak) of experimental data has been designed for small group consumption and is mainly aimed at simultaneous visualization of a small amount of data. The ML data access patterns fundamentally differ from traditional data access patterns. The typical MDSplus database is increasingly showing its limitations. We developed a new data management system suitable for tokamak ML research based on experimental advanced superconducting tokamak (EAST) data. The data management system is based on MongoDB and hierarchical data format version 5 (HDF5). Currently, the entire data management has more than 3000 channels of data. The system can provide highly reliable concurrent access. The system includes error correction, MDSplus original data conversion, and high-performance sequence data output. Furthermore, some valuable functions are implemented to accelerate ML model training of fusion, such as a bucketing generator, the concatenating buffer, and distributed sequence generation. This data management system is more suitable for fusion ML model research and development than MDSplus, but it cannot replace the MDSplus database. The MDSplus database is still the backend for EAST tokamak data acquisition and storage.**

***Index Terms*— Data management, experimental advanced superconducting tokamak (EAST), machine learning (ML), tokamak.**

I. Introduction

EXPERIMENTAL data-driven machine-learning (ML) approaches have been successfully applied to solve various problems in the tokamak community. These problems include data-driven physical models [1], [2], [3], [4], [5], disruption prediction [6], [7], [8], [9], magnetic field control [10], surrogate model [11], [12], [13], [14], [15], [16], experimental data analysis [17], [18], [19], [20], discharge modeling [21], [22], experimental workflow optimization [22], [23], [24], [25], and magnetic field reconstruction [26]. Generally, the data access mode for ML workflows fundamentally differs from the conventional access mode for magnetic fusion experiments or simulation studies. The conventional database (i.e., MDSplus[1]) of Magnetic Confinement Fusion (MCF) has been designed for a small group in the control room, and its primary purpose is to visualize small amounts of data simultaneously [28]. In significant contrast, ML data access modes are driven by algorithms that read and use large amounts of data. Furthermore, data cleaning, normalization, and generation are critical components of successful fusion ML research. These fusion ML research key problems are outlined in the report of the workshop on advancing fusion with machine learning [29]. In particular, the report supports a new fusion data platform intended for ML research. The platform development includes a more suitable data management system design for fusion ML research and development.

The current common tokamak database, MDSplus, does not meet the needs of the tokamak community for ML research. The data-driven ML approaches require large amounts of data and have to process long sequences with different lengths of inputs, whereas typical simulation approaches [30], [31] do not require such sizable experimental data inputs. Specifically, in the fusion recurrent neural network (FRNN) [6], one of the most famous disruption prediction works, 8959 shots were used, which would be unimaginable in the typical simulation code development. Except for the requirements of large amounts of data, fusion ML research requires high input–output (I/O) data read/write, MapReduce [32] metadata calculation, data alignment, data conversion, sequence partitioning, error correction, sequence concatenating, and distributed operation support. MapReduce is a programming model and an associated implementation for processing big datasets with a parallel, distributed algorithm on a cluster. These operations are not supported by the existing MDSplus database of tokamak [33].

To meet the new requirements of the experimental data-driven fusion ML research, we propose a new data management system that combines MongoDB and HDF5 [34] files and develops some backend engines more suitable for tokamak fusion ML research. MongoDB is suitable for data retrieval and preprocessing, while the entry of MongoDB is limited to 64 MB. The hierarchical data format version 5

Manuscript received 30 August 2022; revised 6 November 2022; accepted 17 November 2022. This work was supported in part by the National Key Research and Development project under Grant Y65GZ10593, in part by the National Magnetic Confinement Fusion (MCF) Energy Research and Development Program under Grant 2018YFE0304100, and in part by the Comprehensive Research Facility for Fusion Technology Program of China under Grant 2018-000052-73-01-001228. The review of this article was arranged by Senior Editor M. Kovari. *(Corresponding authors: Chenguang Wan; Jiangang Li.)*

Chenguang Wan, Xinghao Wen, and Xi Deng are with the Institute of Plasma Physics, Chinese Academy of Sciences, Hefei 230031, China, and also with the Science Island Branch, Graduate School of University of Science and Technology of China, Hefei 230026, China (e-mail: chenguang.wan@ipp.ac.cn; wenxh@mail.ustc.edu.cn; xi.deng@ipp.ac.cn).

Zhi Yu, Xiaojuan Liu, and Jiangang Li are with the Institute of Plasma Physics, Hefei Institutes of Physical Science, Chinese Academy of Sciences, Hefei 230031, China (e-mail: yuzhi@ipp.ac.cn; lxj@ipp.ac.cn; j_li@ipp.ac.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TPS.2022.3223732.

Digital Object Identifier 10.1109/TPS.2022.3223732

[1]MDSplus is a set of software tools for data acquisition and storage and a methodology for management of complex scientific data [27].

0093-3813 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hefei Institutes of Physical Science. Downloaded on December 07,2022 at 06:42:42 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

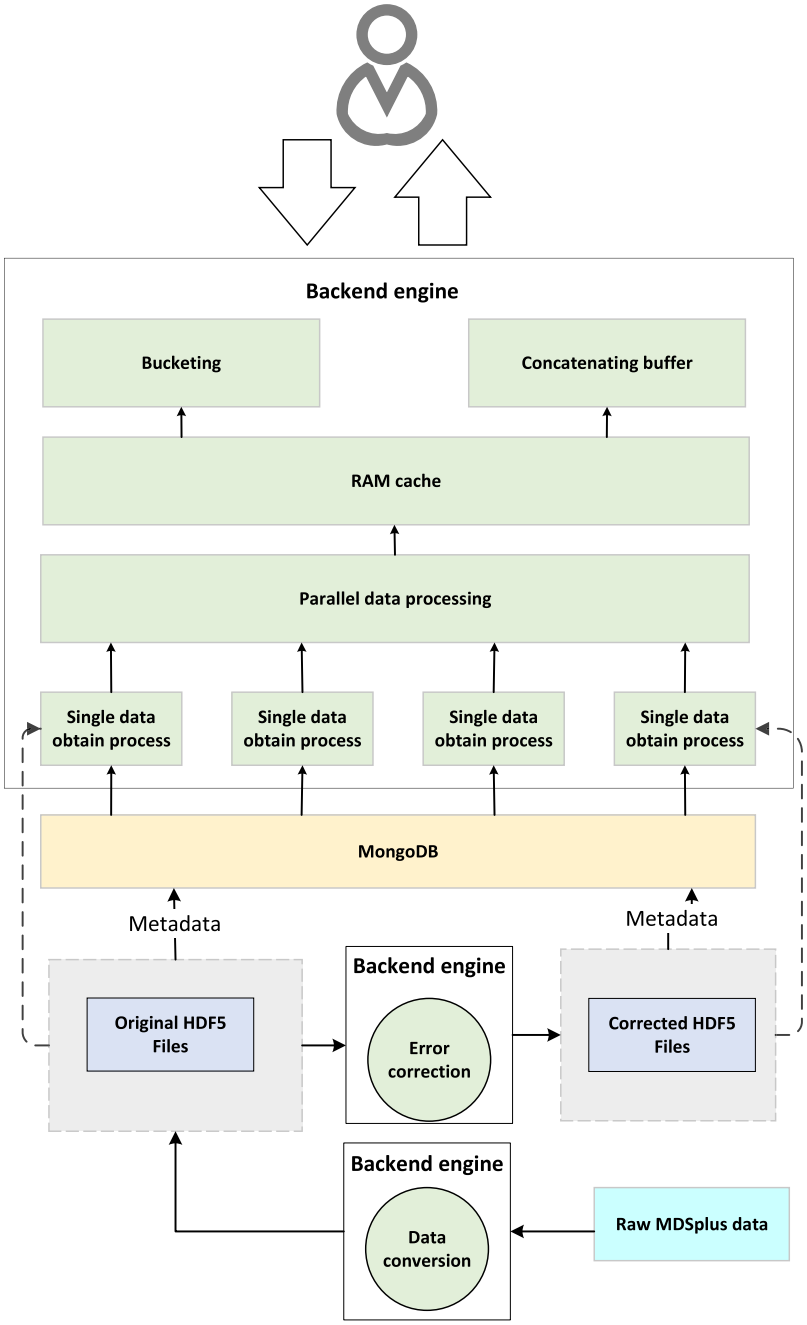


Fig. 1. Data management system architecture. The architecture is divided into three parts: the HDF5 format low-level original data (blue boxes), the metadata managed using MongoDB (yellow box), and the high-level data operation engines (light green boxes).

(HDF5) is designed to store and organize large amounts of data, and it is also suitable for high I/O data access since HDF5 files are file-level data management. The file-level data management features also cause HDF5 files are hard to retrieve and preprocess. We design a hybrid database that combines the advantages of MongoDB and HDF5.

The rest of this article consists of four parts. Section II details the data management system design, including low-level data organization and high-level operation engines. Section III shows the performance comparison between the conventional MDSplus database and this work. Finally, a brief discussion and conclusion are given in Section IV

II. System Design

A. Architecture

The system is designed based on a combination of MongoDB and HDF5. The data management system is divided into three parts: the low-level original data management, the metadata management, and the operational engines. Fig. 1 shows our data management system workflow. The yellow box is metadata management, the blue boxes are low-level original data, and the light green boxes are data operational engines. The main operations are listed as follows.

1) The MDSplus tree data is converted shot-by-shot to HDF5 files.

Authorized licensed use limited to: Hefei Institutes of Physical Science. Downloaded on December 07,2022 at 06:42:42 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

2) Correct the original HDF5 files with the error correction engine.
3) MongoDB contains metadata about the HDF5 files, such as their location, node statistical properties, and so on.
4) The single process is used to read an HDF5 file.
5) Parallel processing is called by bucketing generator, concatenating buffer, and calculating the global metadata.
6) Set bucketing generator (explained further in Section II-C) boundaries.
7) Get aligned sequences from the bucketing generator.
8) Set concatenating buffer time step δ.
9) Get concatenated sequences from the buffer.

B. Database Design

The MDSplus database is designed as a tree structure to store the complex tokamak experimental data. The MDSplus database is unsuitable for fusion ML data accessing and processing. However, recently, experimental data-driven ML research in tokamaks is becoming increasingly popular, and these methods need high robustness, parallelism, and high I/O support of the database or the data management system and also require the database to have the ability to compute global metadata in the distributed MapReduce way.

Our architecture is designed based on MongoDB and HDF5 to meet the new requirements of fusion ML research on tokamaks. MongoDB is a source-available cross-platform document-oriented database program. Classified as a NoSQL database[2] program, MongoDB uses JSON-like[3] documents with optional schemas and supports high I/O. MongoDB also with the ability for high-performance data accessing. Although MongoDB has lots of advantages, on the one hand, even with the latest MongoDB, the maximum entry size is only 64 MB. One tokamak experiment usually produces over 1 GB of data. On the other hand, if each entry had a considerable size, the retrieval performance and robustness would be bad. We use HDF5 files saved as low-level source data storage to solve this problem. The HDF5 is an open-source file format that supports large, complex, heterogeneous data. HDF5 uses a "file directory" structure that allows users to organize data within the file in many different structured ways, as you might do with files on your computer. The HDF5 format also allows for metadata embedding, making it self-describing. The HDF5 is robust and supports high I/O since it is only a file stored on disk, not a server layer required. Our approach is a hybrid architecture of MongoDB and HDF5. The approach uses MongoDB as the data indexing layer and stores metadata in the MongoDB database. As shown in Table I, the metadata includes the discharge duration time, every signal mean, variance, existence flag, correction flag, and so on. The statistical summary is used for data filtering, and the correction flag is used to mark whether the corresponding HDF5 node is corrected. The original MDSplus data is converted and saved on original HDF5 files on the cluster disk.

[2]The NoSQL (aka "not only structured query language") database is a nontabular database and stores data differently than relational tables.

[3]JSON is an open standard file format that uses readable text to store data objects consisting of attribute-value pairs and arrays.

TABLE I

MongoDB Entry Structure. ⟨Node⟩ is a General Name for EAST Diagnostic Signal. It Can Be Replaced by Plasma Stored Energy (WMHD), Ne, and so on

Key	type	meaning
shot	int	Tokamak experiment shot number
file_location	string	The corresponding HDF5 file location
discharge_time	float	Discharge duration time
<node>_existence	bool	Signal existence of this shot
<node>_corrected	bool	Signal correction flag
<node>_mean	float	Signal mean
<node>_stDev	float	Signal standard deviation
<node>_start	float	Node start time
<node>_end	float	Node end time

C. Engine Design

We have developed some operation engines for this data management operation because this hybrid data management system is not easy to operate directly. The engines have six main components: Single data-obtaining process, parallel data processing, bucketing generator, concatenating buffer, error correction, and data conversion. The single data-obtaining process is used to read a shot of data from the hybrid data management system, and the different signals data will be aligned with an identical time axis. Users define the time axis start and end, and the default value "start" is equal to zero, and "end" is equal to the discharge end time. The parallel data-obtaining process calls the single data-obtaining process to get multiple shots of data. The bucketing generator sets some parameters that fit the entire discharge research sequence generation of fusion ML research and caches some data in memory to solve the different speeds from data generation and ML model training. Concatenating buffer concatenates sequences from different shots to generate the aligned subsequences. The error correction is used to check for the invalid value (NaN) and infinity value (Inf) read from the original data. In this step, if the input data has NaN or Inf will be replaced by a linear interpolation value and 3.2×10^{32} (it is not the maximum value of the float 32 type, but it is large enough; and, it can still be calculated without overflowing), respectively. If the node is corrected, the corrected flag will be set as true. The data conversion is used to convert the raw MDSplus data to HDF5 files.

The mini-batching gradient descent [35], [36] is a general technique that helps enhance graphics processing unit (GPU) performance [37] and accelerates the training convergence of ML models. The loss gradients are computed for several examples in parallel and then averaged. The ML model training of the tokamak's experimental data is difficult to use the mini-batching since the tokamak experiment is all time sequences and has different sequence lengths because of the different duration of each tokamak experiment. For the ML approach to work efficiently, the ML model training for forward and backward passes of each gradient computation must be equal for all the examples computed in parallel. This is not possible if different training examples have different lengths. Therefore, there are three options for ML model training of experimental data of tokamaks: 1) using time-slicing techniques, ML models input time slices instead of sequences; 2) time sequence

Authorized licensed use limited to: Hefei Institutes of Physical Science. Downloaded on December 07,2022 at 06:42:42 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

padding techniques, padding sequences with specific values to the same length; and 3) time sequence concatenating. Option 1 is straightforward to implement, so the present work does not give the corresponding interface. A method called a "bucket generator" was developed in the engine to satisfy the requirements of Option 2. For Option 3, we developed an approach based on a sequence concatenating buffer to meet this requirement.

D. Bucketing Generator Design

Tokamak experiment durations vary from experiment to experiment and generate sequences of different lengths. A mini-batch must train on data of the same length, and the short sequences are padded. Therefore, if the sequences are shuffled, a mini-batch could have wildly different maximum and minimum sequence lengths and be action on a lot of padded data. This increases training time and memory usage.

Bucketing is a partitioning algorithm that is used to speed up the training time of a long sequence dataset. We assume there is a set of sequence $S = \{s_1, s_2, \ldots, s_n\}$ is our train set. $l_i = |s_i|$ is the length of sequence s_i. And we use a mini-batch approach to train an ML model for that train set. Each GPU processes a mini-batch sequence in a synchronized parallel manner, so a mini-batch $I_{\text{batch}} = \{s_1, s_2, \ldots, s_k\}$ cost time is proportional to $O(\max_{i\in 1,\ldots,k} l_i)$. Therefore, the processing time of the entire train set is expressed as

$$T(S) = O\big(n/k \times \max_{i\in 1,\ldots,k} l_i\big). \tag{1}$$

If the sequences of the train set were shuffled randomly before mini-batch generation, mini-batch's minimum and maximum sequence lengths would be very different. As a result, the GPU would do useless work for processing the meaningless padding tails of shorter sequences. Additionally, the long sequences with a bit bigger batch size would reach the GPU memory capacity limit. Specifically, using the same mini-batch size for long-sequences input as for short sequences input takes up more GPU memory than expected. We develop a customized bucketing generator backend to optimize the batch training to overcome this flaw and reduce training time. The entire sequence dataset is partitioned into B buckets by the lengths, where each bucket contains data with a similar sequence length. Let $S_i = \{s_{j_1}, s_{j_2}, \ldots, s_{j_{k_i}}\}$. For every bucket, we perform the mini-batch training with different batch sizes. The processing time of the whole set is expressed as

$$T(S) = \sum_{i=1}^{B} O(T(S_i)). \tag{2}$$

The sequences within every bucket are shuffled randomly. And then, the sequences are generated batch sequences batch by batch. To train batchwise with a batch size M, we need M independent shot discharge sequences of the same bucket to feed to the GPU. Zeros pad the different length discharge sequences to the same length. We do this by using M processes to read sequence data in parallel. The M sequences are fed to a buffer first to solve the problem of GPU and central processing unit (CPU) speed mismatch since data from HDF5 files are read through a CPU.

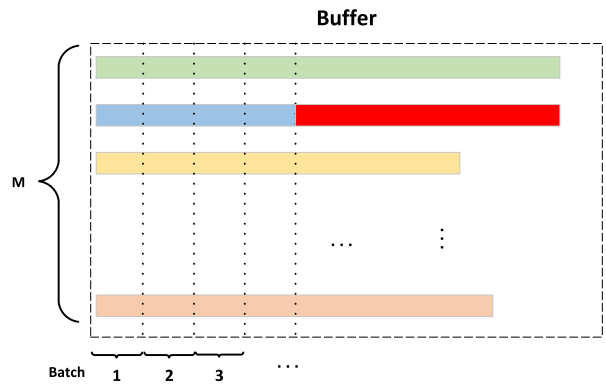


Fig. 2. Sketch of the concatenating buffer engine design. The figure illustrates how batchwise data generate from the buffer with batch size M. The different horizontal bars with different colors represent different shots. A color change in a given row means that a new shot starts. At every time step, the leftmost chunk is cut from the buffer and output. Generally, the concatenating buffer can be regarded as a window moving on the shot sequence set.

E. Concatenating Buffer Design

Another way to handle variable-length time sequence data is concatenating, which is also supported by our data management system engine. Fig. 2 shows the concatenating buffer design. In Fig. 2, a concatenating buffer to output M equal length sequences to feed to the ML model. The sequences come from different shots. First, we need to set a time step δ, before using the concatenating buffer. δ is the minimum time-dependent length. For example, the time-dependent length in disruption research is always set to ~0.1 s [6]. In the buffer, shot sequences are cut into multiples of δ chunks at the beginning. Whenever a shot finishes processing (e.g., the blue shot in Fig. 2), a new shot (red) is loaded. The chunks that are successive in the shot must also be successively generated mini-batches such that the ML model's internal state can be passed correctly. In the concatenating buffer, shot data is not read all at once but is read asynchronously by calling the data reading engine. In practice, there is a data maintenance process that ensures that there is always enough data in the buffer. The concatenating buffer can be regarded as a window that moves through the discharge sequence group. The buffer allows the training of shots of different lengths in batches.

The concatenating buffer data input and the bucketing generator data input are suitable for different ML model training. For example, suppose the entire tokamak discharge modeling is the target of ML research [22]. In that case, the bucketing method is a good choice. A concatenating buffer is a general option if the ML model is to study local time dependencies (like disruption prediction). On the other hand, if the ML model does not require time-dependent, Then the easily slicing approach is more common.

F. Data Conversion and Error Correction

The dataset is selected from the experimental advanced superconducting tokamak (EAST) tokamak MDSplus database original data. Since the MDSplus database cannot support high I/O data accessing, we developed a data conversion engine

Authorized licensed use limited to: Hefei Institutes of Physical Science. Downloaded on December 07,2022 at 06:42:42 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II
COMPARISON OF OUR HYBRID DATA MANAGEMENT AND MDSPLUS

Function	MDSplus	Our hybrid data management system
Demonstrative reading time	$\sim$ 4289 s	$\sim$ 167 s
Bucketing support	False	True
Concatenating support	False	True
High IO concurrency	False	True
ML Algorithm Support	Normal	Good
Visualization support	True	False
Number of plug-ins	Many	None
Diagnostic raw data interface	True	False

(see Fig. 1) to convert MDSplus original data to HDF5 files shot-by-shot directly, and one shot is one HDF5 file. And then, we use the error correction engine to correct the original HDF5 files to get corrected HDF5 files. The high-level engine operates corrected HDF5 files by default (changeable), but we also provide application programming interfaces (APIs) to support custom data error correction or to set it to not correct.

III. PERFORMANCE ANALYSIS

In this section, we compare in detail the performance of our system with the convention database in tokamaks. We compare the bucketing with conventional shuffling and do not compare the concatenating buffer. The concatenating buffer is a solution-suit tool for specific machine learning model training and does not have a counterpart in the conventional fusion database.

A. Comparing Hybrid Data Management and MDSplus

Highly concurrent data accessing directly through the server in EAST's real MDSplus database are difficult to achieve. With so many people working on the EAST's MDSplus database, we cannot perform high-risk operations on it. So, we built a simple MDSplus database for data reading efficiency comparison. The demonstrative database contains 100 signal data from the EAST original shot in the range #74000–76000. We compared the time to read 100 signal data in the shot range #74000–76000 from MDSplus and our data management system. The simple MDSplus and our system were deployed in the same cluster to minimize interference from different environments. Although our model has high concurrence support, we used four processes in both systems for data reading in this test since the MDSplus does not have high concurrence. This means that the data access to our system is faster than reported. The results are contained in Table II. We also listed some common function comparisons in ML, simulation, and experimental research in Table II.

Table II shows that our model has an advantage in ML-related operational support. MDSplus, on the other hand, has the advantage of supporting mainly traditional simulation studies, and it can be directly interfaced with fusion diagnostic systems, which is something our system cannot do at the moment. Our data management is good for fusion ML research but cannot substitute the MDSplus database.

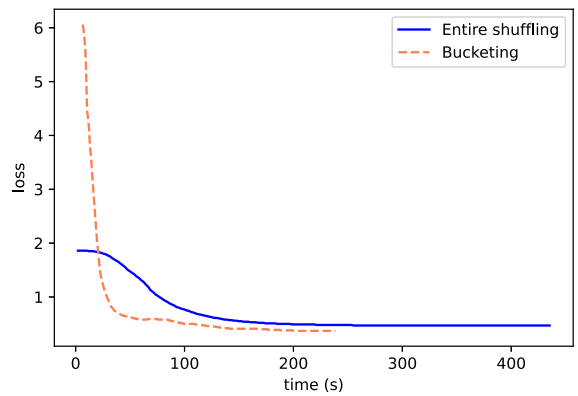


Fig. 3. Training time comparison. The ML model using the bucketing algorithm training converges in about 50 s, while the ML model using shuffling algorithm training converges in about 110 s. This means that the bucketing algorithm is twice as efficient as the common shuffling algorithm in the current task.

B. Bucketing Performance

We tested the ML model's training time in 300 shots sampled from the EAST 2020-2020 campaigns. Fig. 3 compares the bucketing approach and the entire dataset shuffling approach. In this example, we trained a simple single-layer long short-term memory (LSTM) neural network with the same learning rate, optimizer, weights initializer, bias initializer, loss function, and so on. The input parameters come from the experiment data-driven discharge modeling [22], and the output parameter is stored energy W_{mhd}. The bucketing is more efficient than conventional random shuffling in the ML model training. The blue line of Fig. 3 is random shuffling and this approach converges more slowly than the bucketing. We provide a bucketing backend engine that is able to speed up the training of the ML model.

IV. DISCUSSION AND CONCLUSION

As the fusion community is increasing interest in data-driven ML research, the traditional MDSplus database is increasingly showing its limitations in ML research. To fully harness the transformative potential that ML might provide in many fusion energy-related fields, these flaws must be remedied. The idea of fused data platforms (FDP) [29] for ML research is gaining more and more attention. In the present work, a new data management system suitable for fusion ML model research and development of EAST tokamak has been designed and developed to primarily achieve the idea. The system not only accommodates algorithm-driven, high I/O tokamak data access, but also provides a series of advanced interfaces for more efficient ML data generation. The system has the following main functions: data conversion from MDSplus original data, error correction, concatenating buffer sequence generation, bucketing sequence generation, and data statistics. All functions have been developed and have been tested on real fusion ML work about the last closed magnetic surface reconstruction [26].

In future work, the error correction engine will be upgraded by analyzing each tokamak diagnosis and will process the

Authorized licensed use limited to: Hefei Institutes of Physical Science. Downloaded on December 07,2022 at 06:42:42 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

values exceeding the corresponding diagnostic limitation. An automatic data cleaning will be developed to automatically filter out outlier experiments and error experiments. Furthermore, we intend to focus on building a platform to provide an integrated environment for ML and data exploration studies supported by a common interface. Finally, we will test our system on several tokamak databases and develop compatible visualization tools to accelerate data-driven ML research in the fusion community.

Acknowledgment

The authors would like to thank all the members of the experimental advanced superconducting tokamak team for providing such a large quantity of past experimental data. They sincerely thank Prof. Qiping Yuan, Dr. Ruirui Zhang, and Prof. Jinping Qian for explaining the experimental data.

References

[1] E. C. Howell and J. D. Hanson, "Development of a non-parametric Gaussian process model in the three-dimensional equilibrium reconstruction code V3FIT," *J. Plasma Phys.*, vol. 86, no. 1, Feb. 2020, Art. no. 905860102. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0022377819000813/type/journal_article

[2] K. E. J. Olofsson, A. Soppelsa, T. Bolzonella, and G. Marchiori, "Subspace identification analysis of RFX and T2R reversed-field pinches," *Control Eng. Pract.*, vol. 21, no. 7, pp. 917–929, Jul. 2013. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S096706611300049X

[3] D. Moreau et al., "Combined magnetic and kinetic control of advanced tokamak steady state scenarios based on semi-empirical modelling," *Nucl. Fusion*, vol. 55, no. 6, Jun. 2015, Art. no. 063011. [Online]. Available: https://iopscience.iop.org/article/10.1088/0029-5515/55/6/063011

[4] Z. Long, Y. Lu, X. Ma, and B. Dong, "PDE-Net: Learning PDEs from data," in *Proc. 35th Int. Conf. Mach. Learn.*, in Proceedings of Machine Learning Research, vol. 80, J. Dy and A. Krause, Eds., 2018, pp. 3208–3216. [Online]. Available: https://proceedings.mlr.press/v80/long18a.html

[5] B. de Silva, K. Champion, M. Quade, J.-C. Loiseau, J. Kutz, and S. Brunton, "PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data," *J. Open Source Softw.*, vol. 5, no. 49, p. 2104, May 2020. [Online]. Available: https://joss.theoj.org/papers/10.21105/joss.02104

[6] J. Kates-Harbeck, A. Svyatkovskiy, and W. Tang, "Predicting disruptive instabilities in controlled fusion plasmas through deep learning," *Nature*, vol. 568, no. 7753, pp. 526–531, Apr. 2019. [Online]. Available: http://www.nature.com/articles/s41586-019-1116-4

[7] Z. Yang, F. Xia, X. Song, Z. Gao, Y. Huang, and S. Wang, "A disruption predictor based on a 1.5-dimensional convolutional neural network in HL-2A," *Nucl. Fusion*, vol. 60, no. 1, p. 16017, Jan. 2020, doi: 10.1088/1741-4326/ab4b6f.

[8] C. Rea et al., "Disruption prediction investigations using machine learning tools on DIII-D and Alcator C-Mod," *Plasma Phys. Controlled Fusion*, vol. 60, no. 8, 2018, Art. no. 084004. [Online]. Available: http://stacks.iop.org/0741-3335/60/i=8/a=084004

[9] T. Yokoyama et al., "Likelihood identification of high-beta disruption in JT-60U," *Plasma Fusion Res.*, vol. 16, May 2021, Art. no. 1402073. [Online]. Available: https://www.jstage.jst.go.jp/article/pfr/16/0/16_1402073/_article

[10] J. Degrave et al., "Magnetic control of tokamak plasmas through deep reinforcement learning," *Nature*, vol. 602, no. 7897, pp. 414–419, Feb. 2022. [Online]. Available: https://www.nature.com/articles/s41586-021-04301-9

[11] D. J. Clayton et al., "Electron temperature profile reconstructions from multi-energy SXR measurements using neural networks," *Plasma Phys. Controlled Fusion*, vol. 55, no. 9, Sep. 2013, Art. no. 095015. [Online]. Available: https://iopscience.iop.org/article/10.1088/0741-3335/55/9/095015

[12] E. Coccorese, C. Morabito, and R. Martone, "Identification of non-circular plasma equilibria using a neural network approach," *Nucl. Fusion*, vol. 34, no. 10, pp. 1349–1363, Oct. 1994. [Online]. Available: https://iopscience.iop.org/article/10.1088/0029-5515/34/10/I05

[13] C. M. Bishop, P. S. Haynes, M. E. U. Smith, T. N. Todd, and D. L. Trotman, "Fast feedback control of a high temperature fusion plasma," *Neural Comput. Appl.*, vol. 2, no. 3, pp. 148–159, Sep. 1994, doi: 10.1007/BF01415011.

[14] Y.-M. Jeon, Y.-S. Na, M.-R. Kim, and Y. S. Hwang, "Newly developed double neural network concept for reliable fast plasma position control," *Rev. Sci. Instrum.*, vol. 72, no. 1, pp. 513–516, Jan. 2001, doi: 10.1063/1.1323251.

[15] S. Y. Wang et al., "Prediction of density limit disruptions on the J-TEXT tokamak," *Plasma Phys. Controlled Fusion*, vol. 58, no. 5, May 2016, Art. no. 055014. [Online]. Available: https://iopscience.iop.org/article/10.1088/0741-3335/58/5/055014

[16] S. Joung et al., "Deep neural network Grad–Shafranov solver constrained with measured magnetic signals," *Nucl. Fusion*, vol. 60, no. 1, p. 16034, Dec. 2020, doi: 10.1088/1741-4326/ab555f.

[17] C. M. Samuell, A. G. Mclean, C. A. Johnson, F. Glass, and A. E. Jaervinen, "Measuring the electron temperature and identifying plasma detachment using machine learning and spectroscopy," *Rev. Sci. Instrum.*, vol. 92, no. 4, Apr. 2021, Art. no. 043520, doi: 10.1063/5.0034552.

[18] J. E. Lee, P. H. Seo, J. G. Bak, and G. S. Yun, "A machine learning approach to identify the universality of solitary perturbations accompanying boundary bursts in magnetized toroidal plasmas," *Sci. Rep.*, vol. 11, no. 1, p. 3662, Dec. 2021, doi: 10.1038/s41598-021-83192-2.

[19] A. Dinklage et al., "Integrated data analysis for fusion: A Bayesian tutorial for fusion diagnosticians," in *Proc. AIP Conf.*, vol. 988, 2008, pp. 471–480, doi: 10.1063/1.2905117.

[20] R. M. Churchill, B. Tobias, and Y. Zhu, "Deep convolutional neural networks for multi-scale time-series classification and application to tokamak disruption prediction using raw, high temporal resolution diagnostic data," *Phys. Plasmas*, vol. 27, no. 6, Jun. 2020, Art. no. 062510, doi: 10.1063/1.5144458.

[21] C. Wan, Z. Yu, A. Pau, X. Liu, and J. Li, "EAST discharge prediction without integrating simulation results," *Nucl. Fusion*, vol. 62, no. 12, Dec. 2022, Art. no. 126060, [Online]. Available: https://iopscience.iop.org/article/10.1088/1741-4326/ac9c1a

[22] C. Wan, Z. Yu, F. Wang, X. Liu, and J. Li, "Experiment data-driven modeling of tokamak discharge in EAST," *Nucl. Fusion*, vol. 61, no. 6, Jun. 2021, Art. no. 066015, doi: 10.1088/1741-4326/abf419.

[23] N. Nisan and A. Ronen, "Algorithmic mechanism design," *Games Econ. Behav.*, vol. 35, nos. 1–2, pp. 166–196, Apr. 2001. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S089982569990790X

[24] T. Roughgarden, "Algorithmic game theory," *Commun. ACM*, vol. 53, no. 7, pp. 78–86, Jul. 2010, doi: 10.1145/1785414.1785439.

[25] J. Duris et al., "Bayesian optimization of a free-electron laser," *Phys. Rev. Lett.*, vol. 124, no. 12, Mar. 2020, Art. no. 124801, doi: 10.1103/PhysRevLett.124.124801.

[26] C. Wan, Z. Yu, A. Pau, X. Liu, and J. Li, "A machine-learning-based tool for last closed-flux surface reconstruction on tokamaks," 2022, *arXiv:2207.05695*.

[27] *MongoDB Main Page*. Accessed: Sep. 1, 2021. [Online]. Available: https://www.mongodb.com/

[28] R. Anirudh et al., "2022 review of data-driven plasma science," 2022, *arXiv:2205.15832*.

[29] D. Humphreys et al., "Advancing fusion with machine learning research needs workshop report," *J. Fusion Energy*, vol. 39, no. 4, pp. 123–155, 2020, doi: 10.1007/s10894-020-00258-1.

[30] G. Falchetto et al., "The European integrated tokamak modelling (ITM) effort: Achievements and first physics results," *Nucl. Fusion*, vol. 54, no. 4, Apr. 2014, Art. no. 043018. [Online]. Available: https://iopscience.iop.org/article/10.1088/0029-5515/54/4/043018

[31] Y. Li et al., "The simulation of ELM suppression by ion cyclotron resonance heating in EAST using BOUT++," *Nucl. Fusion*, vol. 62, no. 6, Jan. 2022, Art. no. 066043. [Online]. Available: https://iopscience.iop.org/article/10.1088/1741-4326/ac4efd

[32] J. Dean and S. Ghemawat, "MapReduce," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008, doi: 10.1145/1327452.1327492.

[33] F. Wang, Y. Wang, Y. Chen, S. Li, and F. Yang, "Study of web-based management for EAST MDSplus data system," *Fusion Eng. Des.*, vol. 129, pp. 88–93, Apr. 2018, doi: 10.1016/j.fusengdes.2018.02.068.

[34] M. Folk, G. Heber, Q. Koziol, E. Pourmal, and D. Robinson, "An overview of the HDF5 technology suite and its applications," in *Proc. EDBT/ICDT 2011 Workshop Array Databases*. New York, NY, USA: ACM Press, 2011, pp. 36–47. [Online]. Available: http://portal.acm.org/citation.cfm?doid=1966895.1966900

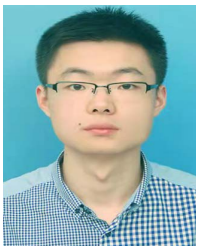
Authorized licensed use limited to: Hefei Institutes of Physical Science. Downloaded on December 07,2022 at 06:42:42 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[35] J. Dean et al., "Large scale distributed deep networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 25, 2012, pp. 1223–1231. [Online]. Available: https://proceedings.neurips.cc/paper/2012/file/6aca97005c68f1206823815f66102863-Paper.pdf

[36] Z. Huang, G. Zweig, M. Levit, B. Dumoulin, B. Oguz, and S. Chang, "Accelerating recurrent neural network training via two stage classes and parallelization," in *Proc. IEEE Workshop Autom. Speech Recognit. Understand.*, Dec. 2013, pp. 326–331. [Online]. Available: http://ieeexplore.ieee.org/document/6707751/

[37] S. Chetlur et al., "cuDNN: Efficient primitives for deep learning," 2014, *arXiv:1410.0759*.



Chenguang Wan was born in Anhui, China, in 1995. He received the B.S. degree in mechanical design from the Hefei University of Technology, Hefei, China, in 2017. He is currently pursuing the Ph.D. degree with the University of Science and Technology of China, Hefei.

His current research interests include tokamak modeling algorithm design and the intersectional field of machine learning and modeling control.

Zhi Yu was born in Tianjin, China, in 1978.

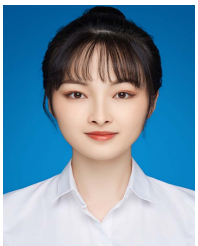
He is currently an Associate Professor in plasma physics at the Institute of Plasma Physics, Chinese Academy of Sciences, Hefei, China. His current research interests include the theory and simulation of plasma physics and the application of data science to Magnetic Confinement Fusion (MCF).

Xiaojuan Liu was born in Anhui, China, in 1987. She received the M.S. degree in applied mathematics from the Hefei University of Technology, Hefei, Anhui, China in 2012.

She is currently a Research Assistant at the Institute of Plasma Physics, Chinese Academy of Sciences, Hefei. Her current research interests include the maintenance of high-performance computing cluster systems, data analysis, and scheduling of distributed workflows in integrated modeling systems.

Xinghao Wen was born in Henan, China, in 1994. He received the B.S. degree in energy and power engineering from the China University of Mining and Technology, Xuzhou, China, in 2017. He is currently pursuing the Ph.D. degree with the University of Science and Technology of China, Hefei, China.

His current research interests include data-driven modeling in tokamaks.



Xi Deng was born in Anhui, China, in 1995. She received the B.S. degree in electrical engineering and automation from Anhui Agricultural University, Hefei, China, in 2017. She is currently pursuing the Ph.D. degree in nuclear engineering with the Institute of Plasma Physics, Chinese Academy of Sciences, Hefei. She majored in nuclear energy science and engineering.

Her current research interests include fault diagnosis of high-power converters.

Jiangang Li was born in Anhui, China, in 1961. He received the B.S. degree from Harbin Engineering University, Harbin, China, in 1982, and the Ph.D. degree from the Chinese Academy of Sciences, Hefei, China, in 1990.

He is currently a Professor at the Institute of Plasma Physics, Chinese Academy of Sciences. His current research interests include tokamak operation and machine learning.

Authorized licensed use limited to: Hefei Institutes of Physical Science. Downloaded on December 07,2022 at 06:42:42 UTC from IEEE Xplore. Restrictions apply.